Interfaces

1. Create an interface with only one method and implement it in a class. Call the method implemented.
   Answer:

```
interface Demo {
    void display();
}
class Test implements Demo {
    public void display() {
        System.out.println("Display Method");
    }
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display();
    }
}
```

OUTPUT:

```
Display Method
```

2. Create an interface with two methods, but implement only one in a class. Call the method implemented.

Answer :

```
interface Demo {
    void display();
    void show();
}
class Test implements Demo {
   public void display() {
        System.out.println("Display Method");
    }
    public void show() {
        System.out.println("Show Method");
    }
```

```
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display();
    }
}
```

3. Use Interface instances to call the implemented method in the implemented class

Answer:

```
interface Demo {
    void display();
}
class Test implements Demo {
    public void display() {
        System.out.println("Display Method");
    }
    public static void main(String[] args) {
        Demo obj = new Test();
        obj.display();
    }
}
```

4. Create two interfaces with one method each. Implement these two interfaces in one class.

Answer:

```
interface A {
    void methodA();
}
interface B {
    void methodB();
}
class Test implements A, B {
```

```
   public void methodA() {
 System.out.println("Method A");
    }
    public void methodB() {
   System.out.println("Method B");
    }
    public static void main(String[] args) {
        Test obj = new Test();
        obj.methodA();
        obj.methodB();
    }
}
```

5. Create two interfaces with the same method (same signature) in both the interfaces. Implement these two interfaces in one class. Call the method.

   Answer:

```
interface A {
    void display();
}
interface B {
    void display();
}
class Test implements A, B {
   public void display() {
        System.out.println("Common Method");
    }
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display();
    }
}
```

6. Create an interface with a default method and implement it in a class. Do not provide implementation to the default method and call the method.
   Answer:

```
interface Demo {
```

```
    default void display() {
        System.out.println("Default Method");
    }
}
class Test implements Demo {
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display();
    }
}
```

7. Create an interface and inherit it from the other interface.

Answer:

```
interface A {
    void methodA();
}
interface B extends A {
    void methodB();
}
class Test implements B {
   public void methodA() {
        System.out.println("Method A");
    }
    public void methodB() {
        System.out.println("Method B");
    }
    public static void main(String[] args) {
    Test obj = new Test();
        obj.methodA();
        obj.methodB();
    }
}
```

8. Create a PUBLIC interface with fields and methods, fields should have values assigned. Implement this interface to some class and print the values of the interface fields and call the interface methods

Answer:

```
public interface Demo {
    int number = 100;
    void display();
}
class Test implements Demo {
   public void display() {
        System.out.println("Display Method");
    }
    public static void main(String[] args) {
        Test obj = new Test();
        System.out.println("Number = " + number);
        obj.display();
    }
}
```

9. Create a PRIVATE or PROTECTED interface and print the values as above scenario

Answer:

Not Possible in Java.

- A top-level interface can only be public or package-private (default).
- It **cannot** be private or protected.

10. Create an interface with private, public and protected fields.

    Answer:
    Not Possible in Java.
    All interface fields are automatically:
    public static final
    You cannot declare them as private or protected.

11. Create an interface with static final variable

Answer:

```
interface Demo {

   static final int NUMBER = 500;
}

class Test {
   public static void main(String[] args) {
      System.out.println(Demo.NUMBER);
   }
}
```

OUTPUT:

500